공통 예제로 이해하는 Flink → Bronze 적재 심화

Flink → Bronze 적재 심화
개인 도메인·테이블 설계 확정

도입 이유 · 내부 동작 · 운영 판단 · 개인 프로젝트 설계

흐름을 먼저 알고 설계한다

8회 단계적 진행 로드맵

STEP 1 · 1회차

개념 & 환경 구성

[핵심 학습]

- OT · 왜 CDC+Lakehouse인가
- CDC/Iceberg 내부 구조와 장단점
- 실습 환경 구성 및 공통 예제 흐름 체험

[프로젝트 진행 방식]

공통 예제 (orders·payments·refunds)로 전체 구조 파악

STEP 2 · 2~3회차

수집·적재 흐름 체득

[핵심 학습]

- 2회차: PostgreSQL → Debezium → Kafka 변경 수집
- 3회차: Flink → Iceberg Bronze 적재·지연 측정

[프로젝트 진행 방식]

공통 예제로 c/u/d·체크포인트·커밋을 직접 다룬 뒤, 3회차에 내 개인 프로젝트 도메인·테이블 설계 확정

STEP 3 · 4~6회차

내 프로젝트 본격 구현

[핵심 학습]

- 4회차: Spark SQL Silver 중복 제거·MERGE
- 5회차: dbt Gold 마트·품질 테스트
- 6회차: 과거 데이터 수정·삭제 소급 정정

[프로젝트 진행 방식]

내가 설계한 도메인 테이블과 지표를 파이프라인에 실제 투입·구현

STEP 4 · 7~8회차

성능 튜닝 & 운영 점검

[핵심 학습]

- 7회차: Iceberg 작은 파일 (rewrite_data_files)·파티션·쿼리 튜닝
- 8회차: Airflow 연동·장애 복구 점검·최종 리뷰

[프로젝트 진행 방식]

Iceberg의 단점(Small Files)을 직접 해결하는 전후 성능 비교 벤치마크 완성 (발표 별도)

공통 예제를 이해하고 내 설계로 가져갑니다

단계	공통 예제에서 할 일	내 설계로 가져갈 것
요구와 구성	결제·환불 변경과 대안 비교	필요한 이력·두 지연 목표
내부 저장	입력 SQL·파일·checkpoint·commit	Bronze 입력과 복구 책임
실제 확인	강사 맥미니 공유 화면 시연	학생 SQL·기대 결과 예측
개인 설계	공통 선택과 한계를 바탕으로 정리	원천 2~3개·입력·검증

수업은 강사 시연·학생 SQL 예측 · 집에서는 공통 전체 실행

공통 도메인 · 주문·결제·환불의 관계

테이블	키와 관계
orders	order_id PK · 주문 한 건
payments	payment_id PK · order_id FK
refunds	refund_id PK · payment_id FK

공통 예제: 결제 100000원 · 같은 환불의 생성·정정·철회

공통 파이프라인에서 오늘 볼 범위

오늘 공통 예제로 확인

후속 회차 · 별도 실행

PostgreSQL
orders
payments · refunds

→

Debezium
WAL → 이벤트

→

Kafka
변경 로그 보관

→

Flink
지속 처리

→

Bronze
Iceberg 원문 append

→

Silver
Spark SQL 현재 상태

→

Gold
dbt + Spark 분석 마트

서로 다른 저장 책임

Flink state 볼륨
입력 위치·pending 복구 상태

Catalog PostgreSQL
현재 metadata 위치 · catalog-db

MinIO / S3
Parquet + Iceberg metadata

오늘: PostgreSQL → Debezium → Kafka → Flink → Iceberg Bronze

왜 CDC·Kafka를 두고 지연을 나눌까요?

CDC·Kafka

변경 전달과 입력 보관
원천·브로커 운영 비용

Bronze 지연

같은 변경 이력이
조회될 때까지

지표 지연

후속 갱신을 거쳐
기대 집계가 읽힐 때까지

수집이 빨라진 것과 최종 지표가 갱신된 것을 나누어 확인합니다

Flink는 Kafka의 변경을 계속 처리합니다

계속 실행하는 SQL

Kafka를 계속 읽고
선택·변환한 결과를
sink로 보냅니다

읽은 위치와 상태

offset과 미확정 작업을
checkpoint에 보관해
복구에 사용합니다

이번 예제의 처리

SQL 조건으로 null 제외
c/u/d 원문과 전달 위치를
Bronze에 추가합니다

Debezium은 변경 수집 · Kafka는 보관 · Flink는 처리 · Iceberg는 테이블 상태 관리

Iceberg는 Parquet를 테이블로 관리합니다

구분	Parquet	Iceberg
관리 범위	컬럼 데이터 파일	파일들을 묶은 테이블 상태
담는 정보	실제 행·컬럼 통계	스키마·snapshot·파일 참조
읽는 대상	지정한 파일의 데이터	선택한 snapshot의 유효 파일
현재 역할	Bronze 행을 저장	파일의 포함·변경을 확정

저장소는 객체 보관 · Catalog는 현재 metadata 위치 · 엔진은 SQL 실행

Flink를 선택한 이유와 운영 부담

선택지	검토할 조건	함께 드는 비용
Airflow + Spark 배치	시간·일 단위 갱신이면 충분	변경 발견·재조회 범위
Spark Structured Streaming	Spark 기반 마이크로배치 활용	trigger·상태·파일 관리
Kafka Connect Iceberg Sink	전달·routing 기능이 요구에 맞음	connector·commit 설정
Flink + Iceberg	지속 처리·입력과 상태 복원	상시 엔진·checkpoint·파일 관리

이번 선택: 지속 적재·상태 복구·Iceberg commit을 연결해 배웁니다

환불 이벤트 한 건이 Flink를 통과하는 과정



생성·수정·삭제 본문은 각각 append · null tombstone 제외 · 현재 상태 계산은 후속 Silver

op=d·tombstone·append를 구분합니다

op=d JSON

삭제 before를 담은 이력
Bronze에 보존

wire-null

payload IS NOT NULL
wire-null value 제외

raw append

c/u/d를 각각 저장
현재 상태는 별도 해석

이번 Debezium envelope 전제입니다 · Bronze의 op=d는 일반 data 행입니다

작업이 멈추면 어디서 다시 시작하나요?

① 멈추기 전

Kafka 입력 위치와
미확정 파일 참조를
checkpoint에 보관

② 상태를 복원

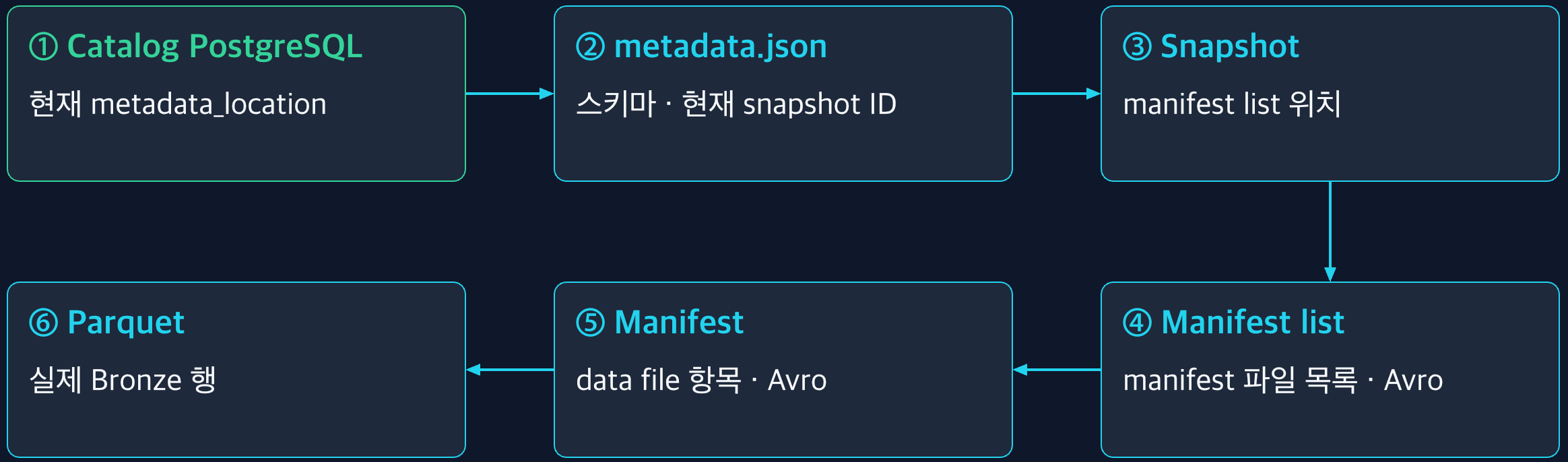
저장한 위치부터 재처리
pending 파일 작업과
commit 기록을 대조

③ 상태 없이 새 job

earliest는 보관 입력 재독
기존 Bronze 중복 적재와
입력 만료를 확인

복구에는 보존된 입력·상태·파일이 필요합니다 · checkpoint 완료와 새 환불 조회는 별도

Iceberg reader가 파일을 찾는 순서



metadata.json · manifest list · manifest · Parquet는 MinIO / S3에 저장

Catalog의 현재 참조가 읽을 metadata와 snapshot을 결정합니다

레코드가 오면 writer는 파일을 작성합니다

계속 들어오는 행

Kafka → filter → writer

Writer의 파일 준비

버퍼·파일 쓰기는 계속 진행

Checkpoint barrier

열린 파일 마감
FlinkWriteResult
(checkpointId, result)

WriteResult를 checkpoint ID와 감싸 전달합니다 · WriteResult 자체에 ID가 있지는 않습니다

Pending 파일 참조를 checkpoint 상태에 보관

아직 테이블에 commit되지 않은 준비 파일일 수 있습니다

연속 파일 작성 → barrier 전 파일 마감 → pending 결과를 복구 상태에 보존

checkpoint 완료 뒤 테이블을 commit합니다

① Checkpoint 완료 통지

복구 상태가 저장됨

② Iceberg committer

준비된 파일·metadata commit

③ Catalog 참조 갱신

현재 metadata 위치 원자적 변경

파일 준비와 테이블 공개는 다른 단계 · 파일 전체가 한꺼번에 업로드되는 트랜잭션이 아닙니다

④ 새로 실행한 Spark SQL 조회

현재 snapshot의 manifest를 따라 포함된 파일을 읽습니다

파일과 metadata 준비 후, 한 테이블의 현재 metadata 참조를 원자적으로 갱신합니다

checkpoint·commit·업무 중복의 경계

Flink checkpoint

복구할 입력과 상태의
지점이 완료

Iceberg commit

새 테이블 상태 확정
복구 시 CP pending과
snapshot commit 기록 대조

업무 중복

다른 전달의 반복은
키·버전 규칙이 필요

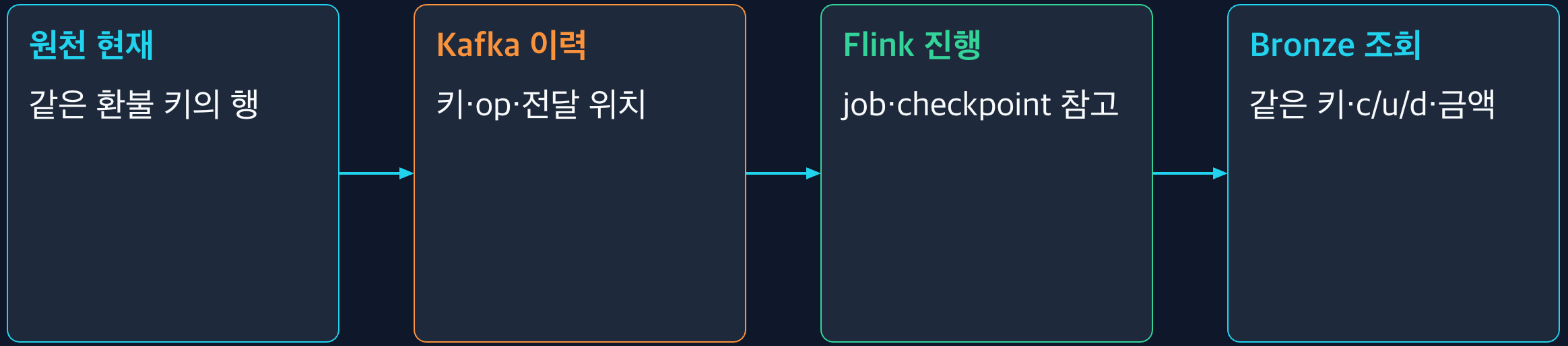
CP 성공과 commit 완료는 별도 · 복구는 pending과 commit 기록 대조

공통 예제에서 기대하는 결과

공통 변경	환불 금액 before → after	확인할 결과
환불 생성 c	null → 20000	같은 환불 키의 생성 이력
환불 정정 u	20000 → 30000	같은 키의 변경 이력
환불 철회 d	30000 → null	d 보존 · tombstone 제외
환불 최종 상태	원천 현재 행 0개	Bronze c/u/d 3이력

시연 전 먼저 예측합니다 · 환불 키는 원천 0행 / Bronze c/u/d 3행

공통 예제는 이 순서로 확인합니다



강사 화면 시연 · 같은 키로 원천·Kafka·Bronze의 실제 결과를 대조합니다

MinIO 위치와 Catalog의 현재 참조

현재 참조

별도 catalog-db
metadata_location

물리 객체

warehouse 버킷
bronze/cdc_events/

복구 상태

별도 flink-state 볼륨
이 버킷과 구분

Catalog 주소 → 사전 Download한 로컬 JSON → MinIO 객체·SQL 대조

파일·JSON·SQL을 교차 확인합니다

확인 대상	물리 객체·관계	SQL에서 대조할 것
현재 table metadata	metadata의 JSON	Catalog metadata_location
snapshot·manifest	JSON 참조 · Avro 목록	snapshots · metadata_log_entries
data 파일	data의 Parquet	files의 경로·행 수·byte 크기
같은 환불 행	그 행을 읽은 파일	payload 키·op · _file

파일 존재 · 현재 참조 · 같은 키 조회 · 물리 bytes 디코드는 서로 다른 증거입니다

payload·_file·파일 행 수의 의미

payload 파싱

before 또는 after의 키
같은 환불·op·금액

_file 경로

행을 읽은 data 파일
이벤트별 1개 보장 없음

파일의 행 수

파일 전체 record_count
키별 matching과 구분

먼저 예측: DELETE 키의 경로와 같은 키의 행 수·op 순서

새 변경 한 건과 지연의 측정 기준

단계·측정	확인할 것	증거의 범위
준비	새 키 · 주문·결제 사전 준비	원천·Bronze 환불 이력 0행
시작	t0 INSERT 호출 · t1 commit 응답	Linux runner의 같은 초시계
끝	t2 같은 환불 행 최초 조회	op=c · 금액 20000 · 결제 키
해석	t2−t0 · t2−t1 두 관측값	반복 조회·쿼리 왕복 포함

강사 화면 시연: 시작·끝을 정하고 같은 환불 키로 확인합니다

운영에서는 입력부터 결과까지 봅니다

범위	볼 증거	질문
입력	rate · pending / lag	입력이 쌓이나
처리	busy · idle · backpressure	어디서 막히나
checkpoint	성공·실패·duration	복구 지점이 진행하나
commit	마지막 성공 경과·소요	테이블 확정이 진행하나
결과	snapshot · 같은 키 SELECT	필요한 변경이 읽히나

backpressure는 느린 하류에서 상류로 전파될 수 있습니다

작은 파일과 유지 관리의 비용

선택·작업	얻는 것·목적	함께 확인할 조건
짧은 checkpoint	잦은 commit 기회	작은 파일·metadata 비용
파일 목표 크기	파일 rolling의 목표	간격·입력량·writer 수
rewrite_data_files	작은 data 파일 재작성	7회차 · 동시 쓰기·연산 비용
Snapshot expiration	과거 table 버전 보존 종료	조회·복구에 필요한 버전
remove_orphan_files	참조되지 않는 객체 정리	pending·복구 안전 보존

현재 snapshot에 없다는 이유만으로 파일을 지우지 않습니다

운영 상황에서 다음 증거 고르기

RUNNING·결과 오래됨

같은 변경·같은 키부터
마지막 확인 경계 찾기

CP 성공·commit 오래됨

로그·Catalog·저장소
가설을 증거로 확인

복구와 보관

입력·상태·파일 보존
보관 범위와 디스크 확인

초기화·offset 변경보다 현재 증거와 보존 상태를 먼저 확인합니다

공통 예제에서 개인 설계로 · 30분

업무와 도메인

누가 어떤 결과로
무엇을 결정하나요?

작은 원천 범위

테이블 2~3개·PK/FK
변경·삭제 의미

입력과 검증

Bronze 보존 계약
같은 키의 기대 결과

내 도메인의 변경 의미와 선택 이유를 학습지에 확정합니다

개인 원천 테이블과 변경 의미

정할 것	학습지에 남길 내용
테이블 2~3개	이름 · 한 행의 업무 의미
키·시간·순서	PK/FK·최신 버전 → 4회차 MERGE
변경·삭제	삭제 정책·소급 정정 → 6회차
지표·튜닝 후보	조인·지표 테스트(5) · 파티션 후보(7)

테이블 이름과 함께 키·변경 사례·해석 기준을 정합니다

Bronze 입력 계약과 검증 기대값

입력 계약

원문·토픽·전달 위치
시각·삭제·보존 정책

두 식별 기준

업무 키는 업무 행
identity는 전달 레코드

검증 한 사례

같은 키의 기대값
MERGE·dbt 테스트 조건

이력 수·현재 건수·업무 지표의 기대를 구분합니다

대표 설계 리뷰와 다음 회차 연결

리뷰 12분

대표 2개 함께 보기 8분
전원 자가 점검 4분

4~6회차 개인 구현

4 Silver 중복제거 MERGE
5 dbt Gold·테스트
6 수정·삭제 소급 정정

7~8회차 튜닝·운영

7 rewrite_data_files
파티션·쿼리 튜닝
8 Airflow·복구·최종 리뷰

8회 모두 수업 · 프로젝트 발표는 별도 일정입니다

공식 근거와 공통 예제 자료

분류	다시 확인할 자료
공통 예제	flink/ingest.sql.template · labs/session01·02
자택 공통 전체 실행	제공 프로그램 · docs/self-study.md
개인 설계	labs/session03 · 설계 학습지·학생 가이드
Flink 1.20	실행·checkpoint·Kafka·backpressure
Iceberg 1.10	구조·JDBC·Flink 쓰기·유지 관리

공식 링크 모음: labs/session03/references.md